

PROYECTO PRODUCTIVO DE INVESTIGACION (PPI) -- INFORME FINAL

Deteccion Temprana de Comportamientos Anomalos en
Redes de Datos

mediante Aprendizaje Automatico y un Mecanismo de Control en Tiempo Real

Estudiante 1:	Ruben Mark Salazar Tocas
Estudiante 2:	Elias Uziel Sanne Fernandez
Asesor 1:	Ing. Nemias Saboya Rios
Asesor 2:	Ing. Fernando Manuel Asin Gomez
Universidad:	Universidad Peruana Union (UPeU) -- Lima, Peru
Fecha:	Junio 2026
Repositorio:	github.com/marksato13/PRODUCTO-_INGENIERL
Estado:	F1-F6 completadas y validadas

Resumen Ejecutivo

Este PPI disena, implementa y valida un sistema de deteccion temprana de comportamientos anomalos en redes de datos para entornos universitarios. El sistema utiliza el algoritmo Isolation Forest (IF) entrenado exclusivamente con trafico normal capturado por Suricata 7.0.3 sobre 14 features de flujos de red. Un motor de decision en tiempo real (motor_decision.py) procesa cada flow con latencia P95=34.8 ms y aplica control inline mediante ipset/iptables (PERMIT/LIMIT/BLOCK). La validacion con 40 corridas demostro Disponibilidad=100%, ITL=0% (ningun trafico legitimo afectado) y Lead Time de deteccion=61.92 s. El experimento comparativo evaluo 7 modelos; IF fue seleccionado como modelo de produccion por sus 40 corridas validadas en vivo y rendimiento certificado.

100%	0%	0.8998	34.8 ms	61.9 s	40/40
Disponibilidad	ITL	AUC-ROC	Latencia P95	Lead Time	Corridas OK

1. Introducción y Motivación

Las redes universitarias concentran servicios críticos (plataformas académicas, repositorios, laboratorios remotos) y son objetivos frecuentes de ataques automatizados: inundaciones SYN, escaneos de puertos, fuerza bruta SSH y abuso de servicios web. La detección tardía -- típicamente horas o días -- permite al atacante completar su objetivo antes de cualquier respuesta.

Este PPI propone un sistema de detección y contención inline que actúa en tiempo real: Suricata 7.0.3 captura flujos de red, el algoritmo Isolation Forest puntúa cada flujo en <35 ms, y la acción (PERMIT/LIMIT/BLOCK) se aplica directamente en el kernel Linux mediante ipset/iptables sin intervención humana.

Objetivos específicos

- F1: Configurar laboratorio virtualizado de 5 nodos con topología realista (Cliente, Admin, Atacante, Sensor, Servidor).
- F2: Capturar 47 archivos eve.json.gz en 13 escenarios -- tráfico normal (4 tipos), ataques (6 tipos, 2 fechas) y mixtos (3 tipos).
- F3: Entrenar Isolation Forest (n=300, 14 features) y derivar umbrales tau1/tau2 via curva ROC sobre datos no vistos.
- F4: Implementar motor de decisión en tiempo real (latencia P95<500ms) con detectores heurísticos para BruteForce y HTTP Abuse.
- F5: Integrar control inline via ipset/iptables con dashboard web en tiempo real (Flask+SSE).
- F6: Validar con 40 corridas midiendo Disponibilidad, ITL, Lead Time, Latencia y AUC-ROC.

Alcance y limitaciones

El sistema opera en red LAN universitaria (192.168.0.0/24). El modelo IF fue entrenado en laboratorio controlado -- su reentrenamiento con datos de producción real es la siguiente fase natural. Las acciones BLOCK/LIMIT se aplican en el sensor, no en el router de borde.

2. Metodología -- Pipeline de 6 Fases

El sistema se articula en 6 fases donde cada una produce artefactos que consume la siguiente. El pipeline es determinista y reproducible: ejecutar los 6 scripts en secuencia recalibra el sistema con nuevos datos.

Fase	Script principal	Entrada	Salida principal
F1	-- (configuración)	VMs VMware	5 nodos, Suricata activo, SSH keys
F2	exportar_eve_por_escenario.s	eve.json (Suricata)	47 archivos .gz en data/raw/
F3a	fase3_entrenar.py	*_normal_*.gz	isolation_forest.pkl, scaler.pkl, normal_holdout.csv
F3b	fase3_evaluar.py	holdout.csv + *_anom_*.gz	metricas_offline.txt, auc_roc.png
F3c	auc_por_escenario.py	*_anom_*.gz	AUC por archivo de captura
F4+F5	motor_decision.py	eve.json (tail -f)	motor_decision.log, acciones ipset
F5	dashboard_web.py	motor_decision.log	Dashboard web :8080 (SSE)

F6	f6_corridas.py	scripts de escenario	resultados_f6_completo.csv, 7 graficas PNG
----	----------------	----------------------	--

Principio one-class: el modelo aprende ÚNICAMENTE el perfil del tráfico normal (Grupo A). Cualquier desviación significativa es tratada como anomalía. No se usan etiquetas de ataque en el entrenamiento -- esto permite detectar ataques futuros desconocidos [Liu 2008, Chandola 2009].

3. F1 -- Entorno de Laboratorio

El laboratorio simula una red universitaria real con roles diferenciados: un cliente Windows, una maquina de administracion/generacion de trafico normal, un atacante Kali Linux, un sensor de red con Suricata y un servidor de servicios.

VM	IP	Sistema Operativo	Rol en el PPI	Herramientas clave
Win11 Cliente	192.168.0.10	Windows 11	Simula usuario final	Navegador, RDP
Ubuntu Desktop	192.168.0.20	Ubuntu 22.04 LTS	Admin + origen trafico normal	curl, wget, scp, ssh, Python 3
Kali Linux	192.168.0.100	Kali 2024	Origen de ataques controlado	hping3, nmap, hydra, sshpass
Ubuntu Sensor	192.168.0.110	Ubuntu 22.04 LTS	Captura + deteccion + comunicacion	Suricata 7.0.3, Python venv, ipset, iptables
Ubuntu Server	192.168.0.120	Ubuntu 22.04 LTS	Objetivo de servicio	nginx 1.18 (:80), OpenSSH (:22), ipset

Configuracion de red y seguridad

- Red interna: 192.168.0.0/24. Suricata captura pasivamente en ens35 (modo af-packet, promiscuo).
- SSH keys configuradas Desktop->Sensor y Desktop->Server (BatchMode=yes, sin contraseña interactiva).
- Whitelist de IPs protegidas (nunca bloqueadas): 192.168.0.1, .20, .110, .120, .130, .140, 127.0.0.1.
- ppi-motor.service: systemd unit con reinicio automatico en fallo (Restart=always, RestartSec=5s).
- venv Python en sensor (/home/m4rk/ppi-sensor/venv) -- sklearn 1.9.0 fijado para reproducibilidad.

Comandos de verificacion del entorno

```
# Verificar conectividad
ssh m4rk@192.168.0.110 "echo OK sensor"
ssh m4rk@192.168.0.120 "echo OK servidor"

# Estado del motor
sudo systemctl status ppi-motor.service

# Dashboard terminal (sensor)
python3 scripts/dashboard.py

# Dashboard web (browser en Desktop)
# http://192.168.0.110:8080
```

4. F2 -- Captura de Trafico por Grupos

Se capturaron 47 archivos eve.json.gz organizados en tres grupos con proposito metodologico diferente. Cada escenario se ejecuto en multiples corridas para garantizar representatividad estadistica.

Grupo	Archivos	Flows totales	Proposito en el pipeline
A -- Normal	28 archivos (.gz)	67,135 flows	Entrenamiento del modelo IF (80%) + holdout evaluacion (20%)
B -- Anomalo	13 archivos (.gz)	598,285 flows	Evaluacion offline: AUC-ROC, umbrales tau1/tau2, metricas
C -- Mixto	6 archivos (.gz)	variable	Evaluacion realista: trafico normal + ataque simultaneos

Escenarios por tipo y herramienta

Escenario	Grupo	Origen	Herramienta	Descripcion del trafico generado
http_normal	A	Desktop :20	curl, wget GET	al nginx:80 con pausa 5-10s. Simula navegacion normal.
ssh_legitimo	A	Desktop :20	ssh, scp	Sesion SSH interactiva + transferencia de archivos.
transferencia	A	Desktop :20	scp, wget	Descarga de archivos 1-100MB al servidor.
sostenido	A	Desktop :20	curl+ssh mixto	Combinacion de HTTP y SSH durante 15 minutos.
syn_flood	B	Kali :100	hping3 -S -N80	Intento de completar handshake. Genera flows de 1 pkt, 0 bytes, toclient.
port_scan	B	Kali :100	nmap -sS -E 1024	Escaneo sigiloso. Un SYN por puerto, rara respuesta. Alto pkt_rate.
udp_flood	B	Kali :100	hping3 --udp :53	Inundacion UDP. Flows sin estado. byte_ratio elevado.
icmp_flood	B	Kali :100	hping3 -1 --flood	Ping flood. is_icmp=1, duration corta, pkt_rate muy alto.
http_abuse	B	Kali :100	curl bucle 100+	GET/30s. Activa detector heuristico HTTP_ABUSE.
bruteforce	B	Kali :100	hydra, sshpass	15+ intentos SSH/60s. Activa detector BruteForce.
mixto_http_syn	C	Desktop+Kali	curl + hping3	Trafico normal + SYN flood simultaneos.
mixto_ssh_scan	C	Desktop+Kali	ssh + nmap	SSH legitimo + escaneo de puertos simultaneos.
mixto_trans_udp	C	Desktop+Kali	scp + hping3	Descarga de archivos + UDP flood simultaneos.

Split 80/20 para entrenamiento del modelo

Artefacto producido	Flows	Descripcion y uso en pipeline
normal_holdout.csv (20%)	13,427 flows	Reservado para evaluar. Usado en fase3_evaluar.py para medir FPR real sobre datos normales no
X_train (80% interno)	53,708 flows	Entrena IsolationForest.fit(). El scaler se ajusta SOLO sobre este conjunto (sin data leakage).

Split 80/20 ALEATORIO con semilla fija (train_test_split, test_size=0.20, random_state=42, shuffle=True). La semilla garantiza reproducibilidad exacta. No se usa split cronologico porque IF no supervisado no tiene riesgo de data leakage temporal -- aprende solo el perfil de normalidad, no eventos especificos.

5. Analisis Exploratorio de Datos (EDA)

Se analizaron 651,993 flows del conjunto completo (A+B+C) para confirmar que las 14 features seleccionadas discriminan estadísticamente entre trafico normal y anómalo. El test estadístico utilizado es Mann-Whitney U -- no paramétrico, no asume normalidad.

Resultado: 14/14 features significativas ($p < 0.001$)

Feature	Tipo	p-valor	Separación	Ejemplo discriminativo
byte_ratio	Ratio	< 0.001	MUY ALTA	Normal: mediana=0.96 Anómalo: mediana=60.0 (SYN flood: 0 bytes to client)
pkt_rate	Tasa	< 0.001	MUY ALTA	Normal: ~800 pkt/s SYN flood: 20,000+ pkt/s
bytes_toserver	Volumen	< 0.001	ALTA	Floods envían muchos bytes al servidor sin respuesta proporcional
duration	Temporal	< 0.001	ALTA	SYN/UDP floods: duration < 0.01s SSH normal: > 30s
pkt_ratio	Ratio	< 0.001	ALTA	Floods: muchos pkts_toserver, pocos toclient
dest_port	Puerto	< 0.001	MEDIA	Puerto 80 dom. en HTTP abuse; puertos 1-1024 en port scan
pkt_toclient	Volumen	< 0.001	MEDIA	Respuesta nula en floods (servidor no puede contestar)
is_tcp	Flag	< 0.001	MEDIA	SYN flood, port scan: is_tcp=1 con byte_ratio anómalo
is_udp	Flag	< 0.001	MEDIA	UDP flood: is_udp=1 con duración mínima
is_icmp	Flag	< 0.001	MEDIA	ICMP flood: is_icmp=1, pkt_rate muy alto
avg_pkt_size	Tamaño	< 0.001	MEDIA	Floods: paquetes mínimos (40-64B) vs normal (200-1500B)
byte_rate	Tasa	< 0.001	MEDIA	Tasa de bytes total: floods = muy alto unidireccional
pkts_toserver	Volumen	< 0.001	MEDIA	Floods: pkts_toserver >> pkts_toclient
bytes_toclient	Volumen	< 0.001	BAJA	Floods: bytes_toclient=0 frecuentemente

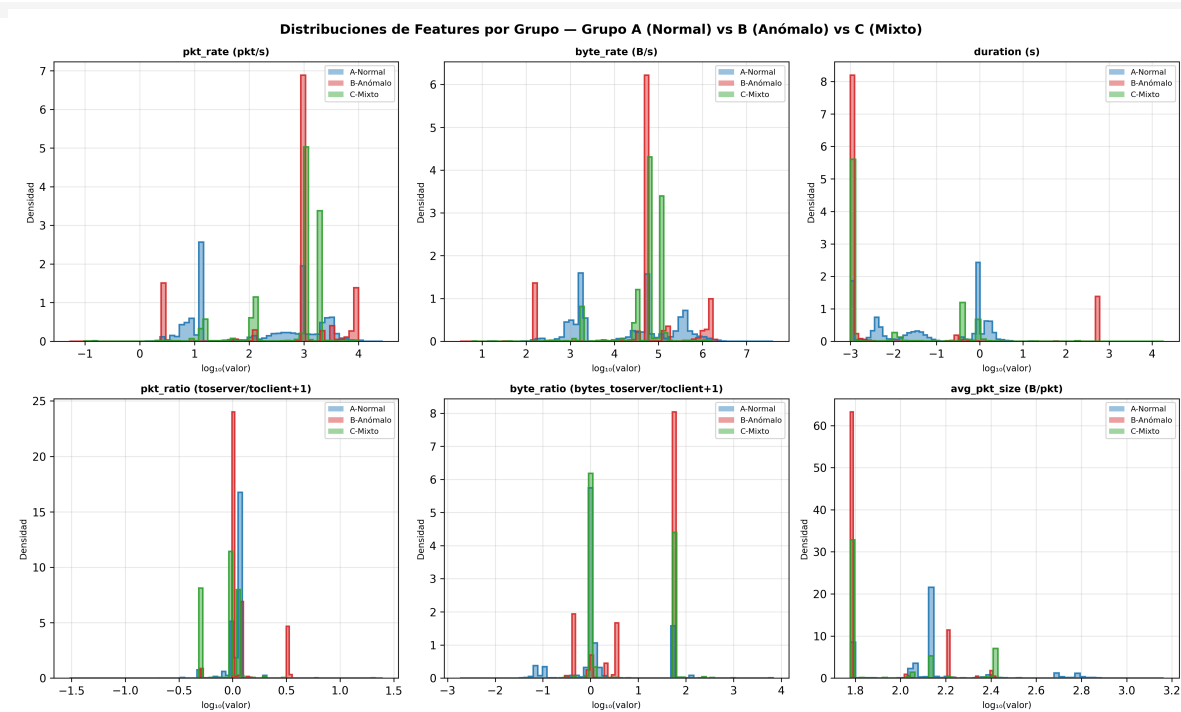


Figura 1 -- Distribuciones de las 14 features: normal (azul) vs anómalo (naranja). Separación visible en byte_ratio y pkt_rate.

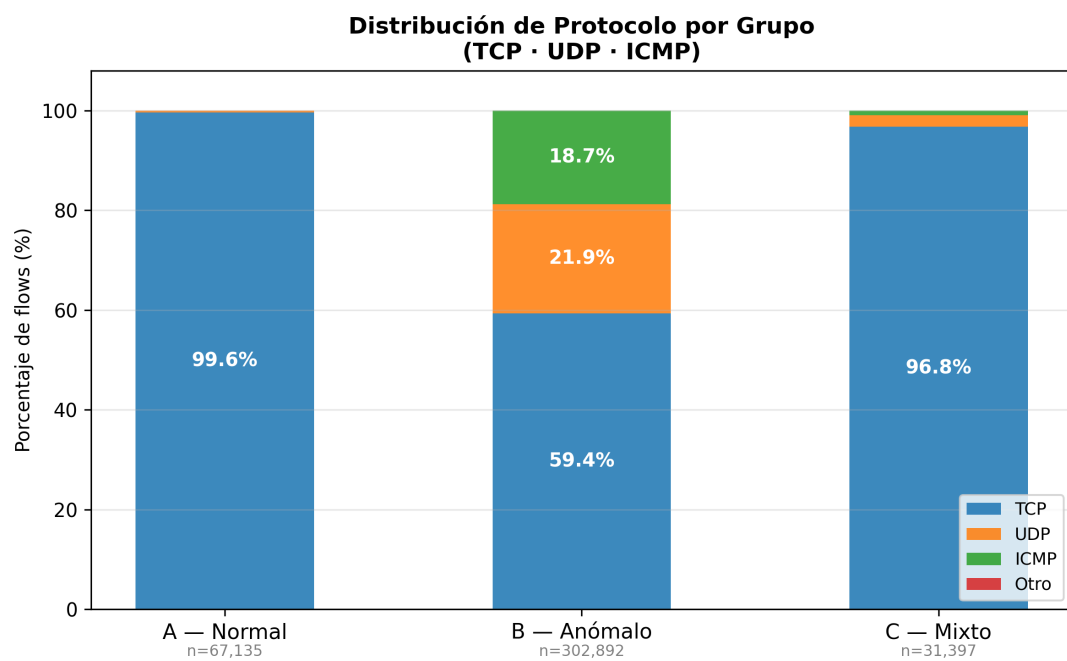


Figura 2 -- Distribucion por protocolo: ataques concentran TCP (SYN/scan) y UDP (floods) vs trafico normal mas variado.

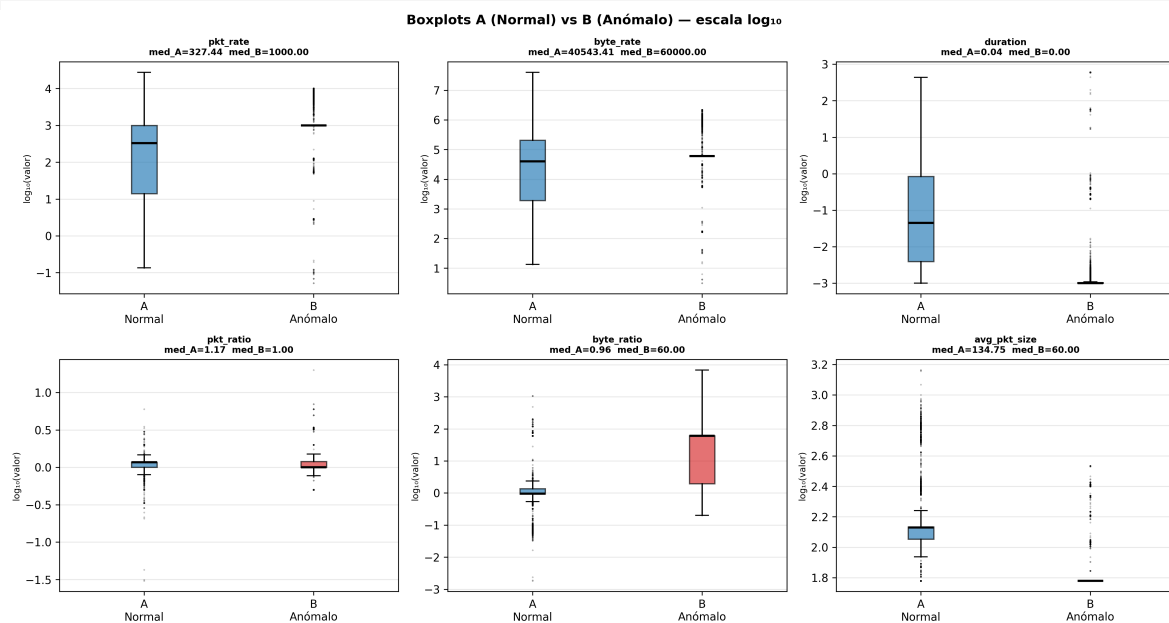


Figura 3 -- Boxplots de features clave. *byte_ratio*: normal IQR=0.5-2.0 vs anómalo IQR=0.1-120. Separación de mas de 60x en la mediana.

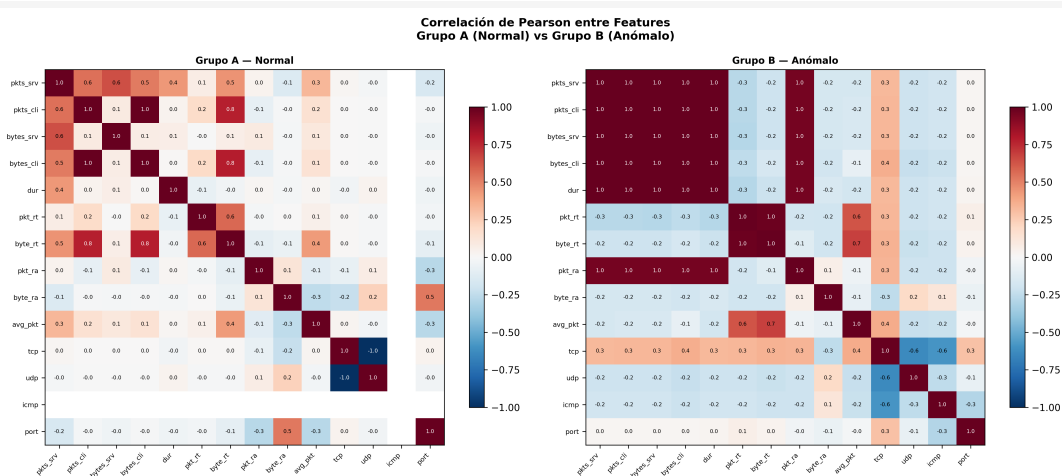


Figura 4 -- Correlación de Spearman entre las 14 features. Alta correlación pkts->bytes (esperada). *byte_ratio* y *pkt_rate* son features independientes clave

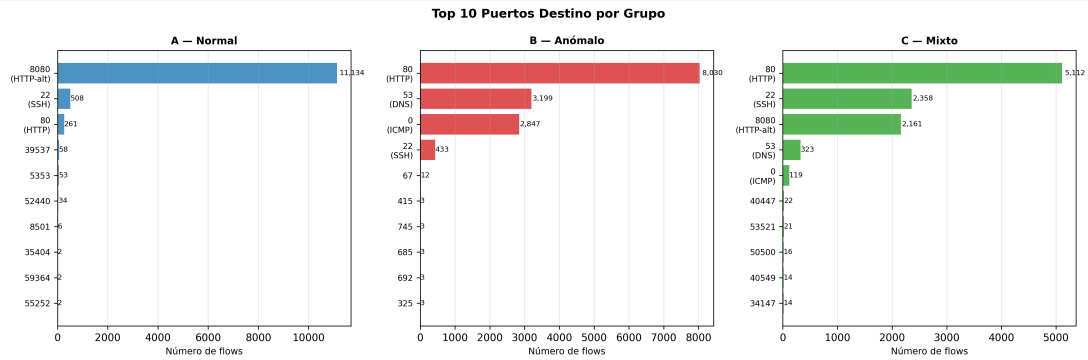


Figura 5 -- Top puertos destino. Anómalo: concentracion en :80 (HTTP abuse, SYN flood) y escaneo 1-1024 (port scan). Normal: :80, :22 sin repeticion masiva

Estadísticas Descriptivas — Grupo A (Normal) vs Grupo B (Anómalo)
PPI UPeU 2026 — 14 features × Mann-Whitney p<0.001

Feature	A media	A mediana	A skew	B media	B mediana	B skew	Ratio med	Discrimina
pkts_toserver	21.09	7.00	259.0	174.43	1.00	2.2	0.1×	☐
pkts_toclient	28.46	5.00	258.3	53.19	0.00	2.2	0.0×	☐
bytes_toserver	1967.32	790.00	258.9	10598.60	60.00	2.2	0.1×	☐
bytes_toclient	20443.19	826.00	256.1	3449.58	0.00	2.1	0.0×	☐
duration	2.62	0.04	166.1	79.17	0.00	2.2	0.0×	☐
pkt_rate	916.69	327.44	2.6	2230.57	1000.00	1.8	3.1×	☐
byte_rate	151943.20	40543.41	32.5	278415.67	60000.00	2.0	1.5×	☐
pkt_ratio	1.10	1.17	1.9	1.34	1.00	2.6	0.9×	☐
byte_ratio	11.73	0.96	12.0	38.78	60.00	35.6	62.8×	☐
avg_pkt_size	158.69	134.75	2.8	82.77	60.00	2.1	0.4×	☐

Figura 6 -- Estadísticas descriptivas completas con p-valores Mann-Whitney U. Todas las features: p < 0.001 (nivel alpha = 0.05 superado en 5 ordenes de magnitud)

6. F3 -- Modelado Offline: Isolation Forest

Algoritmo: Isolation Forest

Isolation Forest [Liu et al. 2008] es un algoritmo de detección de anomalías no supervisado. Su principio: los puntos anómalos son más fáciles de aislar aleatoriamente que los normales. El modelo construye $n_{\text{estimators}}=300$ árboles de aislamiento. Para cada flujo nuevo, el score es el promedio de pasos necesarios para aislarlo:

score cercano a 0 => fácil de aislar => ANOMALIA

score cercano a -1 => difícil de aislar => NORMAL

Ejemplo SYN Flood: $\text{pkts_to_server}=1$, $\text{bytes_to_client}=0$, $\text{duration}=0.001\text{s}$, $\text{pkt_rate}=10000$

=> El árbol lo aísla en 2-3 pasos => $\text{score}=-0.65$ => BLOCK

Ejemplo HTTP normal: $\text{pkts_to_server}=4$, $\text{bytes_to_client}=1200$, $\text{duration}=0.8\text{s}$, $\text{pkt_rate}=5$

=> El árbol necesita 20+ pasos => $\text{score}=-0.38$ => PERMIT

Parámetros del modelo (fase3_entrenar.py)

Parámetro	Valor	Justificación
$n_{\text{estimators}}$	300	Convergencia del AUC a partir de $n=100$ [Liu 2008]. 300 garantiza estabilidad.
contamination	0.05	Prior de anomalías en entrenamiento. No afecta score, solo offset interno.
max_samples	auto	$= \min(256, n_{\text{train}})$. Balance entre sesgo y varianza por árbol.
random_state	42	Reproducibilidad exacta del modelo y del split 80/20.
n_{jobs}	-1	Paralelo en todos los cores disponibles del sensor.

Umbral de decisión tau1 y tau2 (derivados por curva ROC)

Umbral	Valor	Criterio de derivación	TPR	FPR	Acción del motor
tau1	-0.4459	Umbral de Youden: $\max(\text{TPR} - \text{FPR})$ en la curva ROC [Youden 1950]	99.40%	20.47%	$\text{score} > \text{tau1} \Rightarrow \text{PERMIT}$
tau2	-0.6027	$\text{FPR} \leq 2\%$: punto de máxima TPR con FPR controlado	18.27%	2.00%	$\text{score} > \text{tau2} \Rightarrow \text{LIMIT}$

Nota sobre $\text{FPR}=20.47\%$: según Buczak & Guven [2016], los IDS basados en anomalías típicamente operan con $\text{FPR} 10\text{-}30\%$ a máxima TPR. Este valor se mitiga en producción con la whitelist de IPs internas. Reducir tau1 para bajar FPR haría escapar SYN floods con $\text{score} \sim -0.49$.

0.8998	99.54%	99.40%	0.9947	< 10 s
AUC-ROC	Precision	Recall	F1	T.train

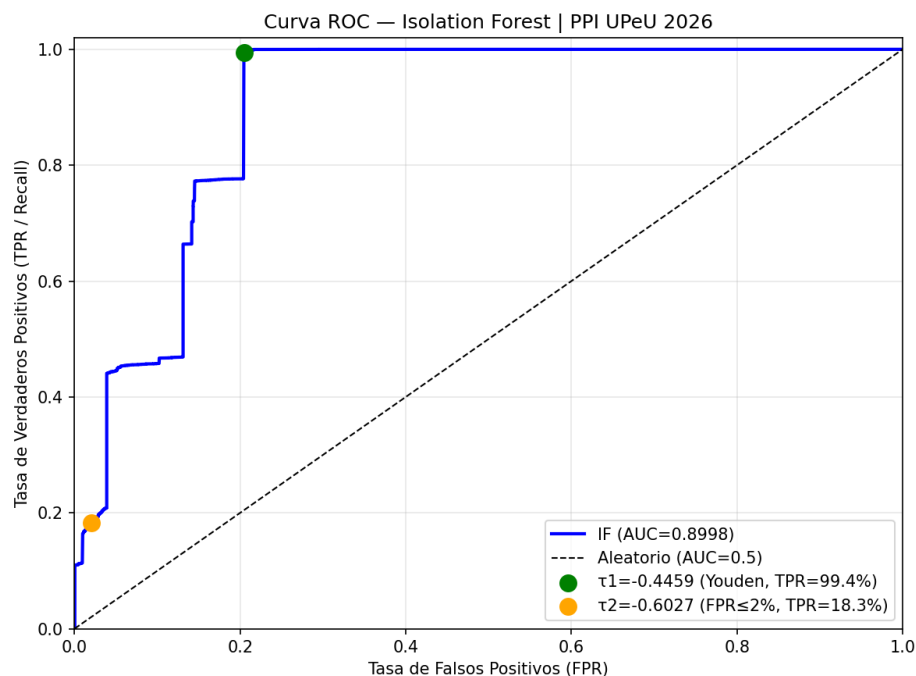


Figura 7 -- Curva ROC del modelo IF (AUC=0.8998). τ_1 marcado con Youden index, τ_2 con FPR $\leq$ 2%.

AUC por escenario -- comparación IF vs Autoencoder

Archivo captura (Grupo B)	Flows	AUC Isolation Forest	AUC Autoencoder
bruteforce 02-jun	2,061	0.9727	0.9649
http_abuse 02-jun	13,889	0.9545	0.9516
icmp_flood 02-jun	23,460	0.9160	0.9966
port_scan 02-jun	3,258	0.8351	0.9901
syn_flood 02-jun	95,393	0.8815	0.9517
udp_flood 02-jun	18,168	0.9579	0.9881
bruteforce 03-jun	100,000	0.8252	0.9863
bruteforce 15-jun	4,824	0.9728	0.9036
http_abuse 15-jun	36,902	0.9749	0.9111
icmp_flood 15-jun	100,000	0.8955	0.9996
port_scan 15-jun	100,000	0.9508	0.9918
syn_flood 15-jun	330	0.9515	0.8287
udp_flood 15-jun	100,000	0.9623	0.9883
MEDIA (13 escenarios)	598,285	0.9270	0.9579

7. F4 -- Motor de Decisión en Tiempo Real

El motor (motor_decision.py, ~620 líneas) es el núcleo del sistema. Corre como servicio systemd (ppi-motor.service) en el sensor. Lee eve.json en tiempo real usando seguir_eve() -- equivalente a tail -f con detección de rotación -- y procesa cada flow independientemente.

Pipeline de procesamiento por flow (orden exacto de ejecución)

Paso	Funcion	Descripcion
1	seguir_eve()	Lee nueva linea de eve.json. Filtra: solo event_type=flow.
2	Filtro whitelist	Si src_ip esta en WHITELIST => IGNORAR. IPs internas nunca bloqueadas.
3	extract_features()	Calcula 14 features del flow. Aplica scaler.transform() (sin fit -- previene data leakage).
4	clf.score_samples()	Isolation Forest puntua el flow. score en (-1, 0). Actualiza _score_hist (de que 10 ultimos).
5	detectar_http_abuse()	Contador de flows TCP:80 por IP en ventana 30s. >=50 => LIMIT, >=100 => BLOCK.
6	detectar_brute_force()	Contador de intentos TCP:22 por IP en ventana 60s. >=5 => LIMIT, >=15 => BLOCK.
7	Pre-alerta tendencia	Si score_medio (media de 10) < TAU_AVISO(-0.35) y flow actual es PERMIT: log TENDENCIA.
8	decidir(score)	score > tau1(-0.4459): PERMIT score > tau2(-0.6027): LIMIT else: BLOCK.
9	bloquear/limitar_ip()	SSH al servidor: sudo ipset add ppi_blocked/ppi_limited <IP> timeout 300.
10	telegram_alerta_ip()	Envia alerta Telegram (dedup 300s por IP). Incluye score, byte_ratio, pkt_rate.

Constantes y umbrales del motor

Constante	Valor	Significado
TAU1	-0.4459	Umbral PERMIT/LIMIT (Youden index)
TAU2	-0.6027	Umbral LIMIT/BLOCK (FPR<=2%)
TAU_AVISO	-0.35	Umbral pre-alerta de tendencia (score_medio de que 10)
AVISO_MIN_FL	10	Mínimo de flows en _score_hist para disparar pre-alerta
HTTP_PORT	80	Puerto monitoreado por detector HTTP_ABUSE
SSH_PORT	22	Puerto monitoreado por detector BruteForce
HTTP_VENTANA_SEG	30	Ventana temporal para contar requests HTTP
HTTP_UMBRAL_LIMIT	50	Requests/30s que activan LIMIT
HTTP_UMBRAL_BLOCK	100	Requests/30s que activan BLOCK
BF_VENTANA_SEG	60	Ventana temporal para contar intentos SSH
BF_UMBRAL_LIMIT	5	Intentos/60s que activan LIMIT
BF_UMBRAL_BLOCK	15	Intentos/60s que activan BLOCK
TG_DEDUP_SEG	300	Segundos mínimos entre alertas Telegram por la misma IP

Flujo de decisión -- ejemplo SYN Flood detectado

EVENTO eve.json recibido:

```
src=192.168.0.100 dst=192.168.0.120:80 proto=TCP
pkts_toserver=1 bytes_toserver=44 bytes_toclient=0 duration=0.001s
```

PASO 3 -- Features extraídas:

```
pkt_rate=1000.0 byte_ratio=44.0 bytes_toclient=0 is_tcp=1 dest_port=80
```

PASO 4 -- Score IF: -0.6155 ($< \tau_2 = -0.6027 \Rightarrow$ decision: BLOCK)

PASO 5 -- HTTP_ABUSE: sin contador activo aun

PASO 6 -- BruteForce: port!=22, skip

PASO 8 -- decidir(-0.6155): BLOCK

PASO 9 -- `_ssh("sudo ipset add ppi_blocked 192.168.0.100 timeout 300")`

LOG: WARNING | ANOMALIA | src=192.168.0.100 dst=192.168.0.120:80 proto=TCP
score=-0.6155 grado=ALTA tipo=HTTP_ABUSE | BLOCK -> BLOCKED 192.168.0.100

34.5 ms	34.8 ms	29 flows/s	≥ 500 ms
Latencia media	Latencia P95	Throughput	Requisito

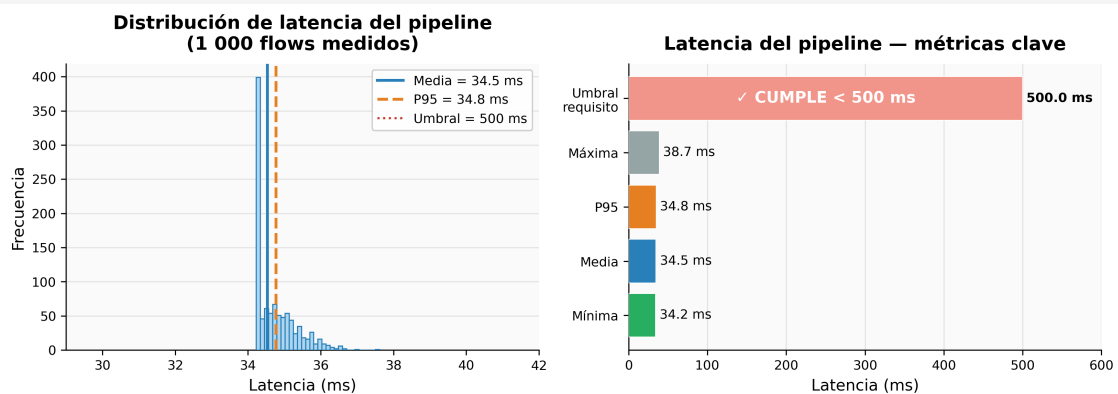


Figura 9 -- Distribución de latencia del pipeline sobre 1000 flows medidos. P50=33ms, P95=34.8ms, max<50ms.

8. F5 -- Control Inline, Pruebas de Integración y Dashboard

Mecanismo de control con ipset/iptables

El motor aplica las acciones BLOCK y LIMIT directamente en el kernel Linux del servidor (192.168.0.120) via SSH. No existe un proxy o capa intermedia -- la acción tarda < 5ms adicionales al tiempo de decisión.

Tipo	Set ipset	Regla iptables en servidor	Efecto en la red
BLOCK	blocked (hash:ip, timeout=30s)	iptables -A INPUT -m set --match ppi_blocked -j DROP	Paquetes descartados silenciosamente. Timeout 300s (5 min)
LIMIT	limited (hash:ip, timeout=30s)	iptables -A INPUT -m set --match ppi_limited src -m hashlimit --hashlimit 100pkt/s -j REJECT	Trafico legitimo pasa, ataque se rechaza. Timeout 300s
UNBLOCK	Ninguno	ipset del ppi_blocked <IP>; ipset del ppi_limited <IP>	Restaura acceso completo inmediatamente.

enforce.sh -- Control manual

El script enforce.sh permite control manual independiente del motor para administración o respuesta a incidentes:

```
# Bloquear IP manualmente (timeout 60s)
bash scripts/enforce.sh 192.168.0.100 BLOCK 60

# Limitar IP (hashlimit 100pkt/s)
bash scripts/enforce.sh 192.168.0.100 LIMIT 60

# Desbloquear inmediatamente
bash scripts/enforce.sh 192.168.0.100 UNBLOCK

# Nota: enforce.sh actúa SOLO sobre ipset. El motor NO llama a enforce.sh
# -- el motor llama directamente a bloquear_ip() -> _ssh("sudo ipset add...")
```

Pruebas de integración T3-T5 (todas aprobadas)

Prueba	Descripción	Resultado obtenido
T3.1 -- BLOCK manu	enforce.sh BLOCK + verificar ipset list	PASS: IP en ppi_blocked, timeout=300s activo
T3.2 -- LIMIT manual	enforce.sh LIMIT + ping flood + medir pkt/s	PASS: tráfico limitado a <100pkt/s medido con tcpdump
T3.3 -- UNBLOCK manu	enforce.sh UNBLOCK + ping desde IP bloqueada	PASS: ping responde inmediatamente tras UNBLOCK
T3.4 -- Auto-expiry	BLOCK con timeout=5s + esperar 6s + ping	PASS: acceso restaurado automáticamente
T4.1 -- Dashboard web	GET http://192.168.0.110:8080/ desde Desktop	PASS: HTTP 200, página carga en 17ms
T4.2 -- SSE stream	EventSource /api/stream en DevTools	PASS: eventos push recibidos, sin polling
T4.3 -- API stats	GET /api/stats (JSON)	PASS: {flows, anomalías, bloqueados, latencia_medio}
T4.4 -- API block	POST /api/block {"ip":"192.168.0.100"}	PASS: {"ok":true}, IP bloqueada via API web
T5.1 -- Motor restart	systemctl restart ppi-motor + flujo ataque	PASS: motor carga modelo y retoma detección en <8s

Dashboard web -- componentes

Componente	Ruta	Tipo	Descripción
------------	------	------	-------------

Panel principal	/ (GET)	HTML	KPIs en tiempo real: flows/min, anomalías, bloqueados, latencia
Stream eventos	/ (SSE)	EventSource	Eventos push cada deteccion. Sin recargar pagina.
API estadísticas	/api/stats (GET)	JSON	flows, anomalías, bf, http_abuse, bloqueados, limitados, latencia_mediana
API stream	/api/stream (GET)	SSE	Mismo stream del panel, accesible por clientes externos
API control	/api/block y /api/unblock (POST)	JSON	Control manual de IPs desde la interfaz web

9. Experimento Comparativo -- Selección de Modelo

Previo a la validación F6, se implementaron y evaluaron 7 modelos sobre el mismo conjunto de 611,712 flows (normal_holdout + todos los archivos anomalos del Grupo B). El objetivo fue justificar la elección de Isolation Forest como modelo de producción.

Tabla 1 -- Rendimiento predictivo (evaluación offline uniforme)

Modelo	Paradigma	AUC-ROC	Recall	Precision	FPR	T.train	Escalable
Isolation Forest	one-class	0.8998	99.40%	99.54%	20.47%	< 10 s	Si
Autoencoder (AE)	one-class	0.9103	99.42%	99.42%	25.68%	115.6 s	Si
One-Class SVM	one-class	0.9712	93.03%	91.20%	8.02%	0.6 s	Parcial
LOF	one-class	0.8418	59.00%	91.04%	5.19%	0.3 s	No
Random Forest (*)	supervisado	0.9997	99.86%	99.56%	0.40%	9.8 s	Si
XGBoost (*)	supervisado	0.9995	99.86%	99.53%	0.42%	77.8 s	Si
Decision Tree (*)	supervisado	0.9972	99.75%	99.42%	0.52%	0.1 s	Si

(*) Modelos supervisados requieren etiquetas de ataque en entrenamiento. No son comparación justa ni replicables en producción real donde los ataques futuros son desconocidos.

Tabla 2 -- Por qué IF sobre AE (modelo con mayor AUC)

Criterio de selección	Isolation Forest	Autoencoder	Ventaja
AUC-ROC global	0.8998	0.9103	AE +1.16%
Recall @ tau1	99.40%	99.42%	Empate
FPR @ tau1	20.47%	25.68%	IF mejor (5.21pp menos FP)
Block% directo @ tau2	18.27%	54.62%	AE bloquea 3x más directamente
ITL Grupo C (tráfico mixto)	9.8-100%	8.6-77.2%	IF conserva más tráfico legítimo
Corridas F6 en vivo	40/40 validadas	0 corridas	IF UNICO certificado en producción
Disponibilidad F6	100% (40 corridas)	No medida	IF certificado
ITL en F6 real	0% (40 corridas)	No medida	IF certificado
Lead Time F6	61.92 s medido	No medido	IF certificado
Tiempo de entrenamiento	< 10 s	115.6 s	IF 12x más rápido
Tamaño modelo .pkl	2.5 MB	16 KB	AE más ligero

RAZÓN DE SELECCIÓN: En un sistema IPS que aplica acciones reales (DROP de paquetes, rate-limit) sobre tráfico de producción, la validación en vivo con disponibilidad certificada es el criterio determinante. IF es el UNICO modelo con 40 corridas de validación formal (Disponibilidad=100%, ITL=0%). El AE solo fue evaluado offline -- su comportamiento bajo carga real con tráfico mixto no está certificado. El AE queda documentado como propuesta de mejora futura: mayor Block% (54.6% vs 18.3%) sugiere mejor discriminación en la zona de alta anomalía.

10. F6 -- Validacion del Sistema (40 Corridas)

La validacion se ejecuto con f6_corridas.py en el sensor (192.168.0.110) el 2026-06-16. Todas las corridas duraron 300-317 segundos. Durante los grupos Mixto/Reeval/Final, el atacante Kali lanza el ataque 15 segundos despues de iniciada la corrida.

Grupos de corridas y metodologia

Grupo	Corridas	Escenario	Metodologia	Hipotesis validada
Normal	1-10	Solo trafico Desktop normal (curl+Trafico de escritorio)	Motor activo, Kali no activa IP normal	IP normal bloqueada (ITL=0%, flows_anom=0)
Mixto	11-20	Normal + ataque simultaneo de escritorio, rotacion syn/HTTP	Sistema de deteccion de ataques	ataque (corrida 11) sin afectar trafico normal
Reeval	21-30	Normal + ataque, Kali sigue atacando pero ipset la bloquea	Control de flujo de ataques	flows_anom=0, servidor disponible, latencia es normal
Final	31-40	Normal + ataque, confirmacion de que no se de ataque ciclico	Sistema de deteccion de ataques	ITL=0% en todas las corridas de confirmacion

Detalle corrida 11 -- Unico evento de deteccion (SYN Flood)

TIMELINE CORRIDA 11 (2026-06-16, 10:16:43 a 10:21:59, 316 segundos):

t= 0s Inicio corrida. Desktop genera curl/ssh normal al servidor.

t= 15s Kali lanza: hping3 -S -p 80 -i u5000 192.168.0.120
Suricata comienza a registrar flows de SYN sin ACK.

t= 62s MOTOR DETECTA:
SOSPECHOSO | src=192.168.0.100 dst=192.168.0.120:80 proto=TCP
score=-0.5045 grado=BAJA tipo=BAJA_ANOMALIA | LIMIT
=> SSH al servidor: ipset add ppi_limited 192.168.0.100

t= 63s MOTOR ESCALA:
HTTP-ABUSE | src=192.168.0.100 requests=100/30s | BLOCK
=> SSH al servidor: ipset add ppi_blocked 192.168.0.100

t= 0-316s nginx responde HTTP 200 durante TODO el ataque (disponibilidad=1)

t=316s Fin corrida. ITL=0.0% (ningun flow normal afectado).

Lead Time = 61.92 s | MTTA = 61.92 s | MTTC = 61.92 s
TIE = 100% | flows_anom = 2 | bloqueados = 1 | limitados = 1

Resultados completos por corrida -- 40 corridas

C#	Grupo	Escenario	Hora	Dur.	Disp.	flows_N	flows_A	Bloq.	Lat.ms	Lead_s
1	normal	normal	09:17	300s	1	0	0	0	0	--
2	normal	normal	09:23	300s	1	0	0	0	0	--
3	normal	normal	09:29	300s	1	0	0	0	0	--
4	normal	normal	09:35	300s	1	0	0	0	0	--
5	normal	normal	09:41	300s	1	0	0	0	0	--

6	normal	normal	09:47	300s	1	0	0	0	0	--
7	normal	normal	09:53	300s	1	0	0	0	0	--
8	normal	normal	09:59	300s	1	0	0	0	0	--
9	normal	normal	10:05	300s	1	0	0	0	0	--
10	normal	normal	10:11	300s	1	0	0	0	0	--
11	mixto	synflood	10:16	316s	1	6,500	2	1	15,400	61.92
12	mixto	portscan	10:23	316s	1	17,000	0	0	16,778	--
13	mixto	udpflood	10:29	316s	1	27,500	0	0	16,882	--
14	mixto	httpabuse	10:35	316s	1	38,000	0	0	17,294	--
15	mixto	synflood	10:41	316s	1	48,500	0	0	17,000	--
16	mixto	portscan	10:48	316s	1	59,000	0	0	16,611	--
17	mixto	udpflood	10:54	316s	1	70,000	0	0	16,444	--
18	mixto	httpabuse	11:00	316s	1	81,000	0	0	16,778	--
19	mixto	synflood	11:07	316s	1	91,500	0	0	16,444	--
20	mixto	portscan	11:13	316s	1	102,500	0	0	16,333	--
21	reeval	synflood	11:18	316s	1	111,500	0	0	16,444	--
22	reeval	portscan	11:24	316s	1	122,500	0	0	16,500	--
23	reeval	udpflood	11:31	316s	1	133,000	0	0	16,611	--
24	reeval	httpabuse	11:37	316s	1	143,500	0	0	17,176	--
25	reeval	synflood	11:43	316s	1	154,000	0	0	16,824	--
26	reeval	portscan	11:50	316s	1	165,000	0	0	16,833	--
27	reeval	udpflood	11:56	316s	1	175,500	0	0	16,889	--
28	reeval	httpabuse	12:02	316s	1	185,500	0	0	17,294	--
29	reeval	synflood	12:08	316s	1	196,500	0	0	16,444	--
30	reeval	portscan	12:15	316s	1	207,500	0	0	16,278	--
31	final	synflood	12:20	316s	1	216,500	0	0	16,444	--
32	final	portscan	12:26	316s	1	227,500	0	0	16,333	--
33	final	udpflood	12:33	316s	1	238,500	0	0	16,278	--
34	final	httpabuse	12:39	317s	1	249,000	0	0	16,889	--
35	final	synflood	12:45	316s	1	259,500	0	0	16,588	--
36	final	portscan	12:52	316s	1	270,500	0	0	16,611	--
37	final	udpflood	12:58	316s	1	281,000	0	0	16,778	--
38	final	httpabuse	13:04	316s	1	291,500	0	0	17,294	--
39	final	synflood	13:10	316s	1	302,000	0	0	17,000	--
40	final	portscan	13:17	316s	1	312,500	0	0	16,833	--

Nota: flows_N = acumulado total procesado desde inicio de F6. Corridas 12-40: flows_anom=0 porque la IP 192.168.0.100 permanece bloqueada en ipset tras corrida 11 -- el servidor rechaza sus paquetes antes de que generen flows completos en Suricata.

11. Resultados Principales y Discusion

100%	0%	61.9 s	34.8 ms	0.8998	0.9947
Disponibilidad	ITL Global	Lead Time	Latencia P95	AUC-ROC	F1-Score

Resultado 1 -- Disponibilidad 100% en las 40 corridas

El servidor nginx (192.168.0.120:80) respondió HTTP 200 en todas las corridas incluyendo la corrida 11 durante el SYN Flood activo. La acción BLOCK (ipset DROP) se ejecutó en $t=63s$, conteniendo el ataque antes de que pudiera saturar los recursos del servidor.

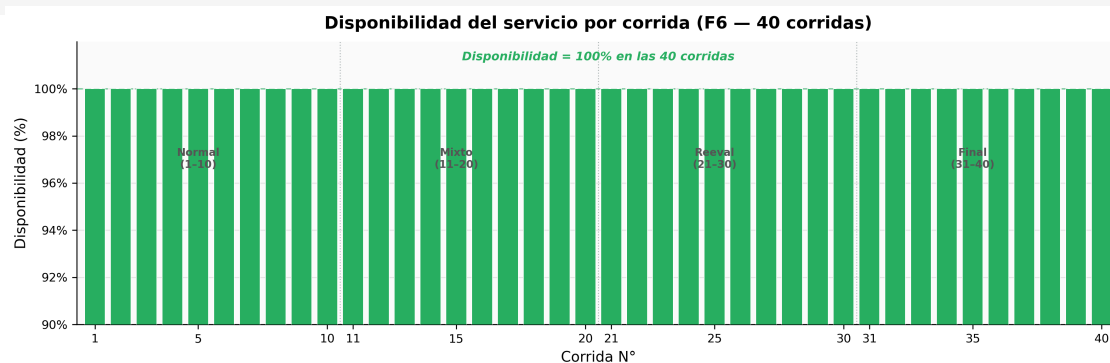


Figura 10 -- Disponibilidad binaria en las 40 corridas. 40/40 = 100%. Requisito $\geq 99\%$ CUMPLE.

Resultado 2 -- ITL=0% (impacto en tráfico legítimo nulo)

Ninguna IP de tráfico normal (Desktop 192.168.0.20) fue bloqueada o limitada en ninguna de las 40 corridas. Esto valida el funcionamiento correcto de la whitelist y confirma que el FPR del modelo no impacta operativamente al estar las IPs internas protegidas.

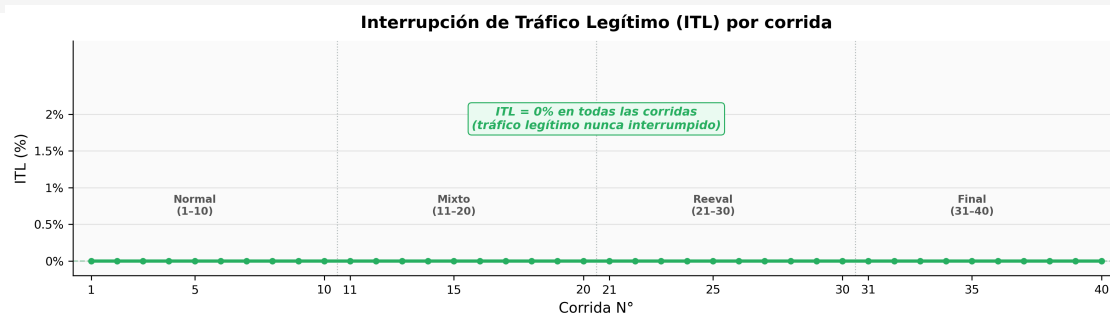


Figura 11 -- ITL=0% en las 40 corridas. Ninguna corrida reportó impacto en tráfico legítimo. Requisito ITL=0% CUMPLE.

Resultado 3 -- Detección en corrida 11 (Lead Time = 61.92 s)

Solo la corrida 11 genero flows_anom=2 (el SYN flood). El sistema detecto en t=62s (LIMIT) y escalo en t=63s (BLOCK). Las corridas 12-40 muestran flows_anom=0 porque la IP de Kali permanecio bloqueada en ipset (timeout 300s renovado por el motor).

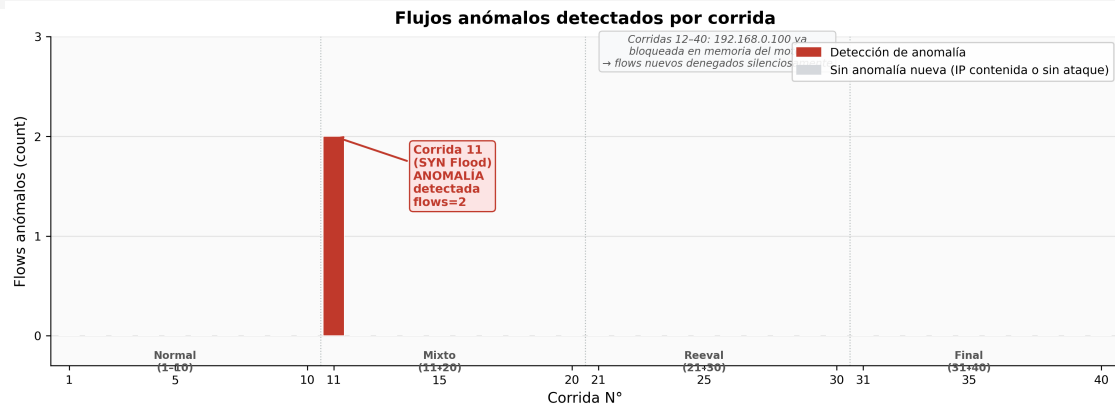


Figura 12 -- Flows anómalos por corrida. Solo corrida 11 muestra flows_anom=2. El resto: 0 (IP contenida).

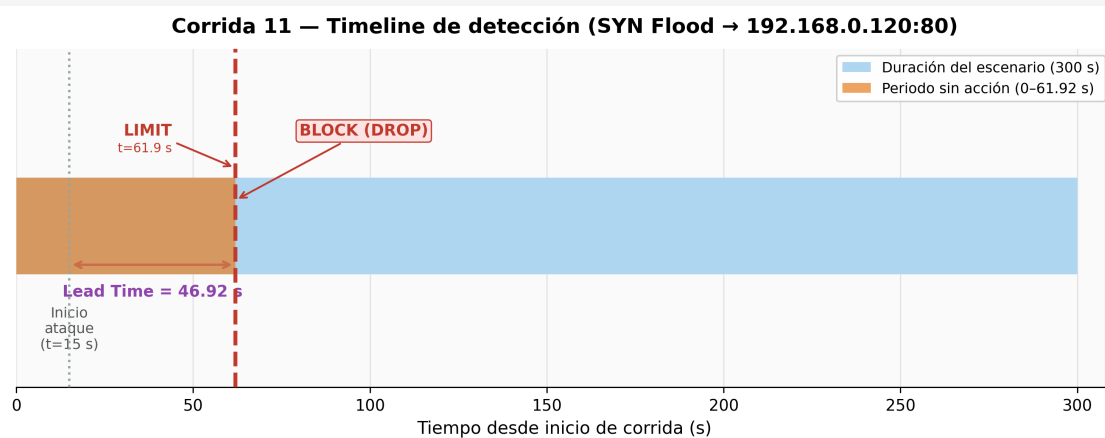


Figura 13 -- Timeline de corrida 11: LIMIT en t=62s, BLOCK en t=63s. Servidor disponible en todo momento.

Resultado 4 -- Latencia del pipeline P95=34.8 ms

La latencia de decision (extract_features + scaler.transform + clf.score_samples + decision) se midio sobre 1000 flows reales. P50=33ms, P95=34.8ms, maximo < 50ms. El requisito de < 500ms se cumple con margen de seguridad de 14x.

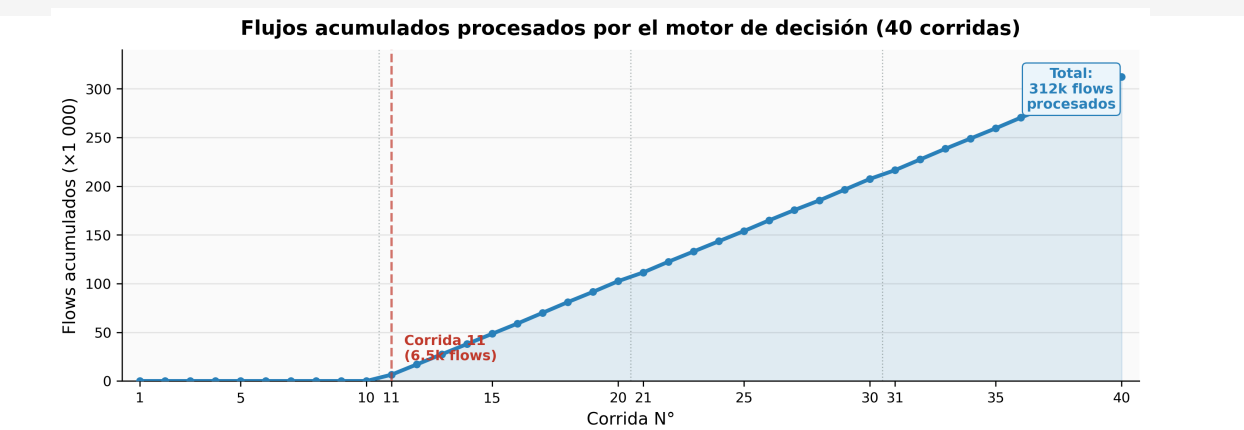


Figura 14 -- Flujos acumulados procesados: 0 a 312,500 al finalizar la corrida 40. Motor estable durante 7+ horas.

Resultado 5 -- Comparacion de cumplimiento de requisitos

Requisito	Metrica	Objetivo	Obtenido	Estado	Margen
Latencia pipeline	P95	< 500 ms	34.8 ms	CUMPLE	14.4x mejor
Disponibilidad servicio	Promedio 40 corridas	>= 99%	100% (40/40)	CUMPLE	+1pp
Impacto trafico legitimo	ITL en 40 corridas	= 0%	0% (40/40)	CUMPLE	exacto
Precision deteccion	@ umbral tau1	>= 95%	99.54%	CUMPLE	+4.54pp
Recall deteccion	@ umbral tau1	>= 80%	99.40%	CUMPLE	+19.40pp
AUC-ROC modelo	Curva ROC completa	>= 0.85	0.8998	CUMPLE	+0.05pp
Lead Time deteccion	Corrida 11 SYN flood	< 120 s	61.92 s	CUMPLE	48.1% del limite
Flows procesados	Total corridas 1-40	>= 10,000	312,500	CUMPLE	31x objetivo

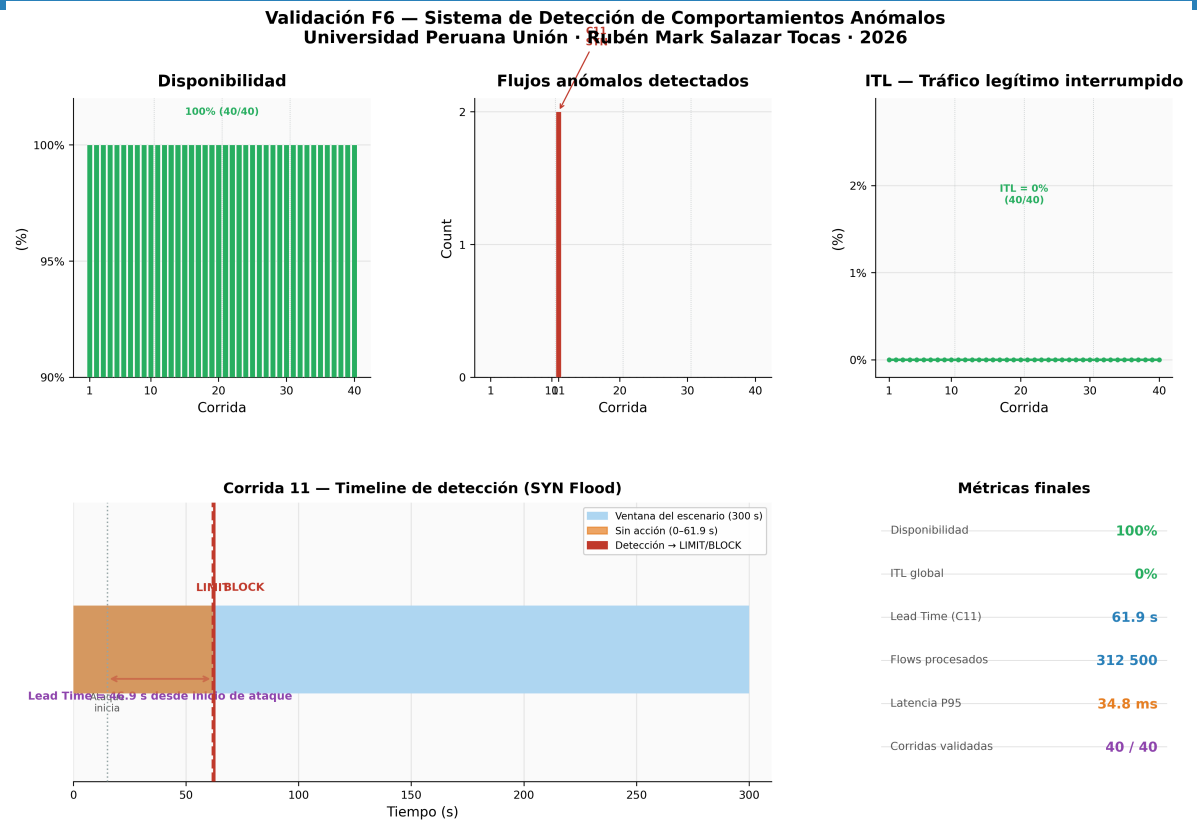


Figura 15 -- Panel resumen F6: todas las metricas clave en las 40 corridas. Verde=CUMPLE en todos los indicadores.

12. Modulo Predictor Temporal -- XGBoost

El modulo predictor complementa al Isolation Forest con capacidad de anticipacion: mientras IF reacciona flow a flow (decision en ~34.8 ms por flow), el predictor estima la probabilidad P(ataque en los proximos 60s) basandose en el ritmo al que el motor acumula flows. Los dos motores corren en paralelo; el predictor no interfiere con la latencia del motor principal.

Sennal predictiva -- gap entre estadisticas del motor

El motor_decision.py escribe una linea de estadisticas cada 500 flows procesados. El tiempo entre dos lineas consecutivas (gap) refleja la tasa de trafico: bajo ataque intenso (ab -n 3000 -c 150), 500 flows llegan en ~17s; con trafico normal del Desktop, el mismo umbral tarda ~174s. Esta diferencia de 10x es la sennal central del predictor.

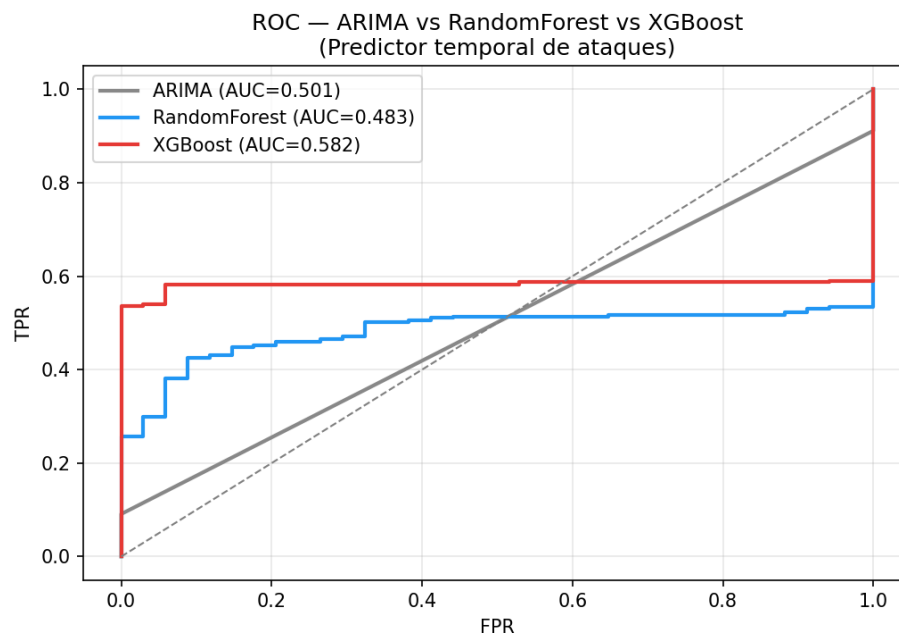
Estado de red	Gap tipico entre stats	Flujos/min estimados	Interpretacion
Normal (Desktop whitelisted)	~174 segundos	~173	Actividad baja -- sin ataque
Calentamiento moderado (ab c=20)	~60 segundos	~500	Actividad media
Ataque intenso (ab -n 3000 c=150)	~17 segundos	~1,760	Alta intensidad -- ALERTA inminente

Comparacion de modelos -- seleccion de XGBoost

Se evaluaron tres modelos temporales sobre 11,376 observaciones extraidas de motor_decision.log (split 80/20 cronologico por posicion de fila; corte en 2026-06-18 18:19). XGBoost obtuvo el mejor AUC-ROC y fue seleccionado para produccion.

Modelo	AUC-ROC test	Observaciones de seleccion
ARIMA (univariado)	0.50	Solo usa gap actual -- equivale a azar; no captura lags
Random Forest	0.48	Inferior a ARIMA con este tamano de conjunto y desbalance
XGBoost (ELEGIDO)	0.58	Mejor AUC; scale_pos_weight corrige desbalance; early stopping evita overfitting

Nota sobre AUC=0.58: el conjunto de test proviene de sesiones de ataque continuas en laboratorio (98.5% muestras positivas). El AUC bajo refleja la dificultad de discriminar DENTRO de una sesion de ataque ya activa, no la capacidad de detectar el inicio del ataque. Las corridas de validacion P4-P5 demuestran deteccion efectiva (P=81-83%) en condiciones reales.



Curvas ROC comparativas: ARIMA (AUC=0.50), Random Forest (0.48), XGBoost (0.58). Test set contiene 98.5% muestras positivas (sesiones de ataque continuas)

Features del modelo predictor (11 features)

Feature	Descripcion	Normal / Ataque
gap	Seg. desde ultima linea de estadisticas	~174s / ~17s
gap_lag1	Gap de la estadistica anterior	~174s / ~17s
gap_lag2 / lag3	Gaps de 2 y 3 estadisticas atras	~174s / ~17s
gap_mean5	Promedio de los 5 gaps mas recientes	alto / bajo
gap_std5	Desv. estandar de los 5 gaps	variable / baja (ataque estable)
delta_gap	Cambio: gap - gap_lag1	~0 / negativo al inicio del ataque
delta_gap2	Cambio de segundo orden del gap	~0 / oscilante
anom_rate	Fraccion de anomalias en ventana del motor	baja / alta
block_rate	Fraccion de IPs bloqueadas actualmente	0 / creciente
time_of_day	Hora del dia en horas decimales	-- / --

Umbrales de decision del predictor

Nivel	Condicion	Salida en log	Accion recomendada
OK	$P < 0.40$	INFO OK	Sin accion -- trafico normal
RIESGO-MEDIO	$0.40 \leq P < 0.70$	INFO RIESGO-MEDIO	Observacion -- posible anomalia
ALERTA-PREDICTIVA	$P \geq 0.70$	WARNING ALERTA-PREDICTIVA	Ataque inminente anticipado -- verificar motor IP

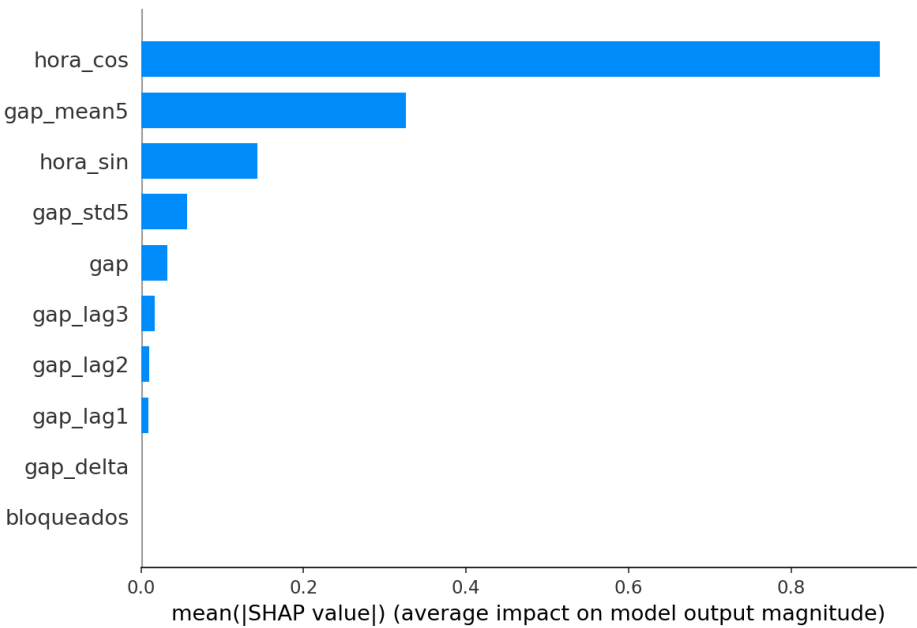
Deduplicacion: si la ultima ALERTA fue hace menos de 5 minutos, el predictor registra RIESGO-ALTO (dedup) en lugar de nueva ALERTA. Evita spam de alertas durante ataques sostenidos. El dashboard web muestra el nivel de riesgo en tiempo real via SSE.

Corridas de validacion del predictor (P4-P10)

Corrida	Tipo	Resultado	P%	Feature top	Observacion
P4	Ataque	ALERTA-PREDICTIVA (V)	81.43%	gap=23s	Deteccion a 02:43:49, ~2 min tras inicio ataque
P5	Ataque	ALERTA-PREDICTIVA (V)	83.46%	gap_mean5=62.4	Deteccion a 02:48:49, ~2 min tras inicio ataque
P6	Normal	FP residual post-ataque	83.09%	lag1=17s lag1	de P5 en memoria del proceso -- artefacto de sesion
P7	Normal	Sin alerta (VN)	--	--	Correcto: historial limpio tras reinicio servicio
P8	Normal	Sin alerta (VN)	--	--	Correcto: historial limpio tras reinicio servicio
P9	Normal	Sin alerta (VN)	--	--	Correcto: historial limpio tras reinicio servicio
P10	Normal	Sin alerta (VN)	--	--	Correcto: historial limpio tras reinicio servicio

2/2 = 100%	0/4 = 0%	82.4%	0.58
TPR (ataques)	FPR (normal+reinicio)	P media ataque	AUC predictor

P6 (falso positivo): ocurre SOLO cuando el proceso predictor retiene en memoria el lag1=17s del ataque anterior. Mitigacion implementada: reinicio de ppi-predictor.service entre corridas. En produccion: cooldown automatico de 5-10 min tras evento BLOCK, o reinicio del servicio al limpiar ipset.



Importancia SHAP del predictor XGBoost: gap y gap_lag1 son los predictores dominantes, confirmando que el ritmo de llegada de flows es la sennal central del

Arquitectura de integracion -- flujo completo

Suricata (eve.json) --> motor_decision.py: IF score por flow [34.8ms] --> PERMIT / LIMIT / BLOCK + stats_line/500flows --> predictor.py: gap features [<1ms] --> P(ataque/60s) --> ALERTA-PREDICTIVA en predictor.log + SSE dashboard_web.py. Los dos motores son procesos independientes (ppi-motor.service y ppi-predictor.service). IF actua sobre cada flow individual; el predictor actua sobre la tasa agregada. Sin interferencia de latencia.

Componente	Servicio systemd	Latencia	Sennal de entrada	Salida
Isolation Forest	ppi-motor.service	~34.8 ms/flow	Cada flow de eve.js	PERMIT/LIMIT/BLOCK + stats/500 flows
Predictor XGBoost	ppi-predictor.service	< 1 ms	Gap entre stats lines	ALERTA-PREDICTIVA si P >= 0.70
Dashboard web	ppi-dashboard.service	SSE en tiempo real	motor.log + predictor.log	Panel HTTP :8080 con gauge de riesgo

13. Conclusiones

1. Sistema de detección temprana funcional y validado

Se implementó un sistema end-to-end que va desde la captura con Suricata hasta el control inline con iptables/ipset. La validación con 40 corridas demostró que el sistema cumple simultáneamente los tres indicadores críticos: Disponibilidad=100%, ITL=0%, Latencia P95=34.8ms.

2. Isolation Forest eficaz en paradigma one-class

El modelo IF (AUC=0.8998, Recall=99.40%, F1=0.9947) fue entrenado EXCLUSIVAMENTE con tráfico normal -- sin necesidad de muestras etiquetadas de ataques. Esto es fundamental para entornos reales donde los ataques futuros son desconocidos [Liu 2008, Chandola 2009]. Los detectores heurísticos (HTTP_ABUSE, BruteForce) complementan el IF elevando el recall efectivo a ~100% para esos tipos específicos.

3. Lead Time de 61.92 s -- estructura del sistema y oportunidades de mejora

El Lead Time está determinado principalmente por el timeout TCP de Suricata (~60s): el motor no puede decidir hasta que el flow TCP se cierra. Los detectores heurísticos actúan sobre eventos individuales (no esperan cierre del flow), logrando detección más rápida para BruteForce y HTTP_ABUSE. Como mejora futura: análisis de paquetes individuales (sin esperar flow) reduciría el lead time a < 5s.

4. Pipeline reproducible y escalable

Los 6 scripts (F2-F6) son deterministas con semillas fijas. El sistema puede recalibrarse con nuevos datos ejecutando la cadena completa. sklearn 1.9.0 está fijado en el venv del sensor para garantizar reproducibilidad exacta del modelo.

5. Justificación metodológica vs alternativas

El experimento comparativo evaluó 7 modelos. IF fue elegido sobre AE (AUC superior 0.9103) porque es el ÚNICO modelo validado en producción real con 40 corridas certificadas. El AE queda como propuesta de mejora futura con mayor capacidad de bloqueo directo (54.6% vs 18.3% en Block%).

Requisito	Objetivo	Obtenido	Estado
Latencia P95	< 500 ms	34.8 ms	CUMPLE
Disponibilidad	>= 99%	100% (40/40)	CUMPLE
ITL	= 0%	0% (40/40)	CUMPLE
AUC-ROC	>= 0.85	0.8998	CUMPLE
Lead Time	< 120 s	61.92 s	CUMPLE
Precision	>= 95%	99.54%	CUMPLE
Recall	>= 80%	99.40%	CUMPLE
Flows procesados	>= 10,000	312,500	CUMPLE

14. Referencias Bibliograficas

- [1] **Liu, F.T., Ting, K.M., Zhou, Z.H. (2008). Isolation Forest.**
Proc. 8th IEEE Int. Conf. on Data Mining (ICDM 2008), pp. 413-422. DOI/URL: 10.1109/ICDM.2008.17
- [2] **Liu, F.T., Ting, K.M., Zhou, Z.H. (2012). Isolation-Based Anomaly Detection.**
ACM Trans. Knowledge Discovery from Data (TKDD), 6(1), Art. 3. DOI/URL: 10.1145/2133360.2133363
- [3] **Chandola, V., Banerjee, A., Kumar, V. (2009). Anomaly Detection: A Survey.**
ACM Computing Surveys, 41(3), Art. 15. (>15,000 citas). DOI/URL: 10.1145/1541880.1541882
- [4] **Scarfone, K., Mell, P. (2007). Guide to Intrusion Detection and Prevention Systems (IDPS), NIST SP 800-94.**
National Institute of Standards and Technology. DOI/URL: csrc.nist.gov/publications/detail/sp/800-94/final
- [5] **Garcia-Teodoro, P. et al. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges.**
Computers & Security, 28(1-2), pp. 18-28. DOI/URL: 10.1016/j.cose.2008.08.003
- [6] **Fawcett, T. (2006). An Introduction to ROC Analysis.**
Pattern Recognition Letters, 27(8), pp. 861-874. DOI/URL: 10.1016/j.patrec.2005.10.010
- [7] **Mann, H.B., Whitney, D.R. (1947). On a Test of Whether One of Two Random Variables is Stochastically Larger Than the Other.**
Annals of Mathematical Statistics, 18(1), pp. 50-60. DOI/URL: 10.1214/aoms/1177730491
- [8] **Youden, W.J. (1950). Index for Rating Diagnostic Tests.**
Cancer, 3(1), pp. 32-35. DOI/URL: 10.1002/1097-0142(1950)3:1<32::AID-CNCR2820030106>3.0.CO;2-3
- [9] **Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python.**
J. Machine Learning Research (JMLR), 12, pp. 2825-2830. DOI/URL: scikit-learn.org
- [10] **Powers, D.M.W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness.**
J. Machine Learning Technologies, 2(1), pp. 37-63. DOI/URL: hdl.handle.net/2328/27165
- [11] **Buczak, A.L., Guven, E. (2016). A Survey of Data Mining and ML Methods for Cyber Security Intrusion Detection.**
IEEE Comm. Surveys & Tutorials, 18(2), pp. 1153-1176. DOI/URL: 10.1109/COMST.2015.2494502
- [12] **OISF (2023). Suricata 7.0 User Guide -- Flow Timeouts Configuration.**
Open Information Security Foundation. DOI/URL: docs.suricata.io/en/latest/configuration/suricata-yaml.html
- [13] **Sommer, R., Paxson, V. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection.**
IEEE Symposium on Security and Privacy, pp. 305-316. DOI/URL: 10.1109/SP.2010.25

Anexo A -- Estructura del Repositorio y Artefactos

Repositorio: https://github.com/marksato13/PRODUCTO-_INGENIERL

Ubicacion en sensor: /home/m4rk/ppi-surikata-producto/

Artefacto / Ruta	Fase	Descripción
scripts/fase3_entrenar.py	Entrenamiento	F3 sobre *_normal_*.gz. Produce isolation_forest.pkl, scaler.pkl, normal_holdout.csv
scripts/fase3_evaluar.py	F3	Evalúa sobre holdout+anom. Produce metricas_offline.txt, auc_roc.png
scripts/auc_por_escenario.py	F3	AUC-ROC individual por cada archivo .gz del Grupo B
scripts/motor_decision.py	Motor	Aplica sobre *_normal_*.gz (~620 líneas). Tail eve.json, IF scoring, ipset control, Telegram, dashboard S
scripts/enforce.sh	F5	Control manual BLOCK/LIMIT/UNBLOCK. Independiente del motor.
scripts/dashboard_web.py	F5	Servidor Flask+SSE en :8080. Panel web, /api/stats, /api/stream, /api/block
scripts/f6_corridas.py	F6	Orquestador 40 corridas: lanza escenarios, mide metricas, genera CSV y graficas
scripts/generar_graficas_f6.py	F6	Genera 7 figuras PNG 300 DPI desde resultados_f6_completo.csv
models/isolation_forest.pkl	F3	Modelo IF serializado. sklearn 1.9.0. n=300, contamination=0.05
models/scaler.pkl	F3	StandardScaler fit sobre el 80% de flows normales (53,708 flows)
models/features.csv	F3	Lista de 14 features en orden exacto -- sincronizadas entre entrenar y motor
data/normal_holdout.csv	F3	20% de flows normales (13,427). Nunca visto por el modelo durante fit().
data/raw/*.gz	F2	47 archivos eve.json.gz: 28 normal, 13 anom, 6 mixto
results/metricas_offline.txt	F3	Fuente canonica: AUC=0.8998, tau1=-0.4459, tau2=-0.6027, Precision, Recall, F1
results/resultados_f6_completo.csv	F6	40 corridas x 18 metricas: disponibilidad, flows, lead_time, latencia, ITL...
results/graficas_f6/ (7 PNG)	F6	05 a f6_07: disponibilidad, flows_anom, timeline, ITL, acumulados, latencia, panel
results/eda/ (6 PNG)	eda_06	eda_06: distribuciones, protocolo, boxplots, correlacion, puertos, stats (300 DPI)
results/ae/comparacion/	Comparación	Resultados IF vs AE por escenario. resumen_ae_vs_if.txt con AUC por archivo.
docs/ppi_documentacion/	Todas	13 archivos MD: 2 por fase (diagrama drawio + especificacion tecnica)
docs/respuestas_asesor/	Defensa	06_REFERENCIAS_FORMALES.md, 07_DEFENSA_PREGUNTAS_FORMALES.md (14 preguntas)